Benchmarking local and commercial LLMs for political science text classification

Hanno Hilbig

2026-04-27

Abstract

I benchmark four local open-weight models against three commercial API models, two from OpenAI and one from Anthropic, on ten political science classification tasks, with 500 items per task. On the cross-task average, the best local model is within 0.002 macro F1 of the strongest API model, with Claude Sonnet 4.6 sitting between them. Top point estimates are split across four of the seven models, and task-level uncertainty is substantial. Local inference is slowest on the largest dense architecture, but the local mixture-of-experts model is the fastest model in the comparison. Batching helps on short prompt tasks with limited accuracy loss at moderate batch sizes, but increases the rate of malformed output on tasks with long, multi-class codebooks. Overall, local open-weight models are competitive on many applied political science classification tasks. However, my results do not support a simple local versus API ranking.

1 Motivation

Political scientists and researchers in adjacent fields who use LLMs for text classification face a practical choice. Commercial API models are reliable, well-documented, and easy to call from R or Python. However, they also cost money per item, send the data off-machine, and create reproducibility risk when the underlying model changes between versions. Local open-weight models running through Ollama or vLLM avoid per-item API charges, keep the data on the researcher's machine, and make exact model versions easier to preserve.

When it comes to evaluating local model performance, the main question is not "are local models as good as API models in general?" but "is the local model good enough on the specific task I want to run?" In this report, I address four practical sub-questions: are local models competitive with commercial ones on common political science classification tasks? How long do they take, and can they run on a single laptop? Does batching speed them up? And how much does the answer vary across tasks?

Across the ten tasks, the top three models (gpt-5.5, Claude Sonnet 4.6, Gemma 4 31B) sit within 0.002 macro F1 of each other on the cross-task average, with per-task winners split across four of the seven models. Local inference can match or beat the commercial API tier on speed, and batching helps on short prompt tasks but raises the rate of malformed output on tasks with long, multi-class codebooks.

2 Design

The benchmark covers ten classification tasks drawn from published political science replication archives, seven models (four local, three commercial API: two OpenAI, one Anthropic), and N=500 items per task sampled with a fixed random seed. The same items are evaluated by each model. I report macro F1, accuracy, Matthews correlation coefficient (MCC), latency, and the rate of malformed output (the share of model responses that could not be parsed into a valid label). The CSV column for the latter is `parse_err_rate`. Macro F1 is the main cross-task summary because it is defined for all tasks and gives each class equal weight, but it is not sufficient on its own for imbalanced label sets, where accuracy and MCC are more informative. All comparisons use 95% paired-by-item bootstrap confidence intervals (1000 iterations).

I ran each model on each item once. Local Ollama calls used temperature 0.1. The commercial API models used provider-default decoding, with `reasoning_effort=medium` for the OpenAI tiers. I sampled items with a fixed random seed (20260422), and the sampling is reproducible. Local inference ran on a single Apple Silicon MacBook (32 GB) through Ollama, without the optional internal-reasoning ("thinking") mode. All four local models were 4-bit quantized (Q4_K_M) variants, so the local results should be read as commodity-hardware inference rather than full-precision performance. OpenAI calls went through the chat completions API with a strict JSON schema. Anthropic calls used tool-use to constrain outputs to the task label schema. I use prompts that are verbatim from the source paper's coding instructions where possible. Where the source paper did not use an LLM, I derive the prompt from the published codebook (per-task provenance is in Table 1).

The serial benchmark includes all seven models. The batched inference comparison includes six (Claude Sonnet 4.6 was added after the batched runs were complete and is included only in the serial benchmark).

3 Results

3.1 Local models are competitive on average

The top three models are essentially tied on average. gpt-5.5 has the highest mean macro F1 (0.621), with Claude Sonnet 4.6 at 0.620 and Gemma 4 31B at 0.619. The spread across these three is 0.002 points. All seven models fall within roughly 0.05 macro F1 of each other (Figure 1), while per-task spread within any single model exceeds that range. Most task-level model differences fall inside bootstrap CI overlap. The conclusion is robust to the metric choice: the same top three sit at the top under mean accuracy and mean MCC, and the rank correlation between mean macro F1 and mean MCC across the seven models is 0.93 (the per-model averages are in the appendix). Among the OpenAI tiers, gpt-5.4-nano is the cost-efficient option: the 5,000 nano predictions cost about $1.20, compared with $21.66 for gpt-5.5, with a 0.021 F1 lag (0.600 vs 0.621).

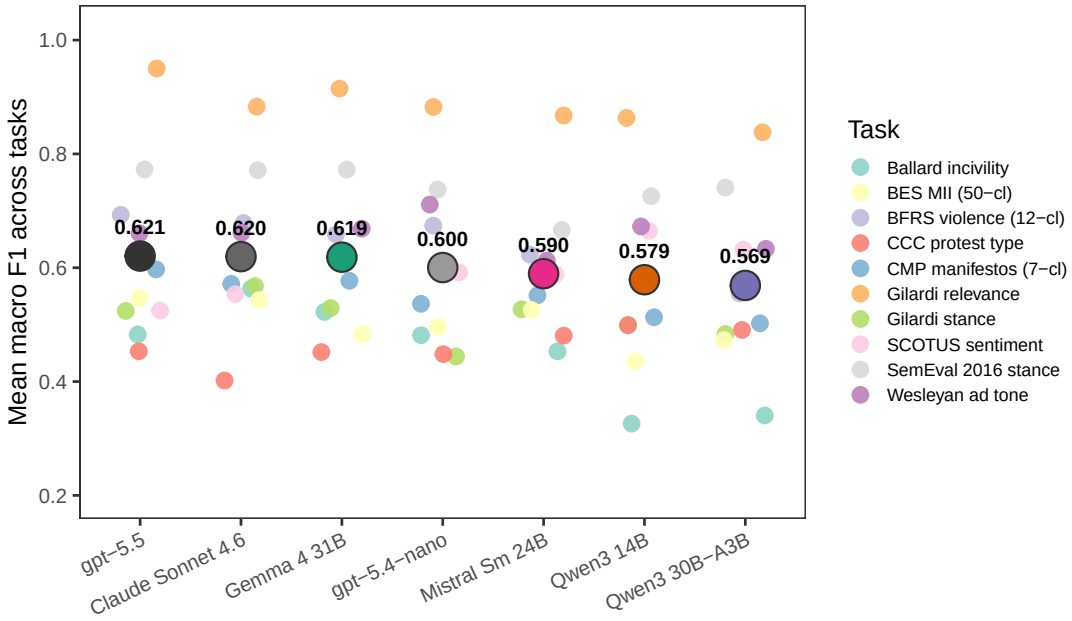

Figure 1: Mean macro F1 across ten tasks per model. Large filled circles show cross-task means (numeric labels); small dots show per-task macro F1 values, coloured by task.

The larger differences are across tasks. At 500 items per task, the bootstrap intervals are wide enough that, on nine of ten tasks, five to seven models have intervals that overlap the top model's interval. Only Gilardi relevance separates a clear top group of two. Task-specific validation matters more than small differences

between highly ranked models.

3.2 No model wins everywhere

Figure 2 aggregates the ten tasks into four families. The family-level averages do not show a simple local versus API pattern. gpt-5.5 is strongest on policy-topic coding. Claude, Gemma, and Qwen3-14B are close on sentiment, stance, and tone. Event coding is close across several models. At the task level, the top point estimates are split across four models: gpt-5.5 leads on five tasks (Gilardi relevance, SemEval stance, BFRS violence, CMP manifestos, BES MII), Claude Sonnet 4.6 on two (Ballard incivility, Gilardi stance), Qwen3-14B on two (Halterman CCC, SCOTUS sentiment), and gpt-5.4-nano on one (Wesleyan ad tone).

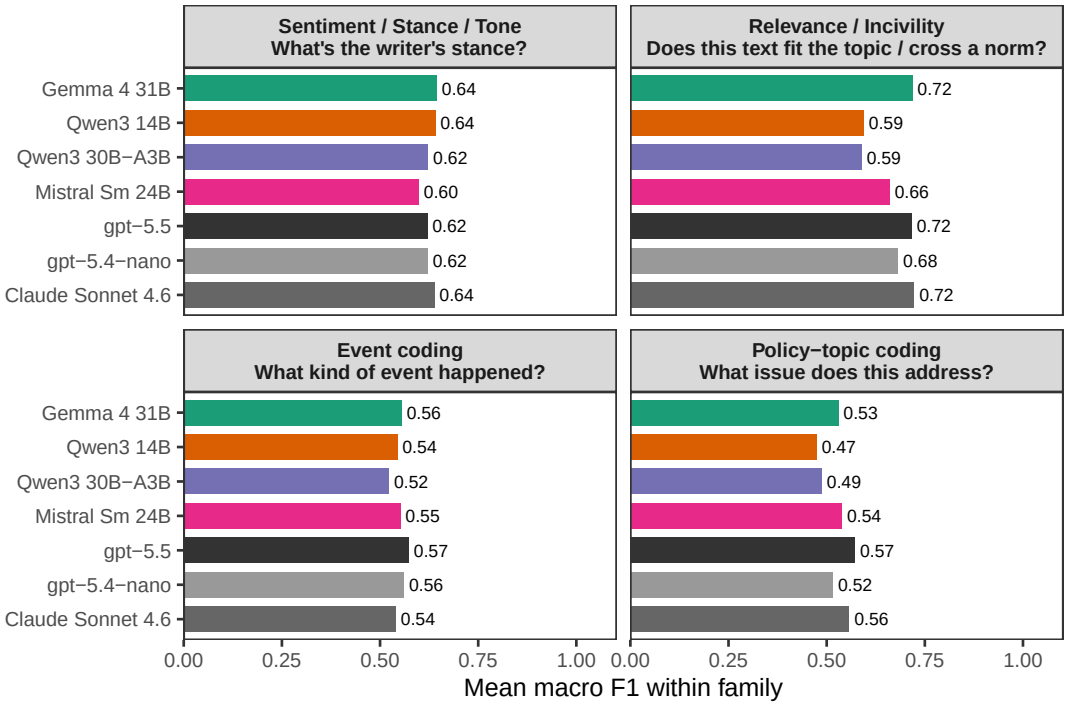

Figure 2: Mean macro F1 within task family, by model. Each family contains two to four tasks; family-level averages are descriptive only.

3.3 Local inference is not uniformly slower

Local inference speed varies sharply with model size and architecture (Figure 3). The fastest model is Qwen3-30B-A3B (a sparse "mixture-of-experts" model that uses about three billion active parameters per call), at about 0.6 s/item. The slowest is Gemma 4 31B, the largest dense local model, at about 4.7 s/item. Qwen3-14B and Mistral Small 24B fall in between. For applied use, parameter count alone is a poor guide to speed: the MoE model is faster than the smaller dense Qwen3-14B.

All four local models ran the full benchmark on a single 32 GB Apple Silicon MacBook; the largest, Gemma 4 31B, fit comfortably in unified memory. Latencies will reorder on different hardware, especially among the local models, depending on memory and compute. For reference, the API tiers in this run sat between roughly 0.7 and 1.8 s/item end-to-end, but those numbers mix model speed with provider serving load, network, and queueing, so they are not directly comparable to local inference time and are not shown in the figure.

3.4 Batching helps on short prompts, hurts on long codebooks

Batched prompting sends multiple items in one call and asks the model to return an array of labels, rather than sending one item at a time. It is a common way to reduce per-item latency for API models (Pipal et

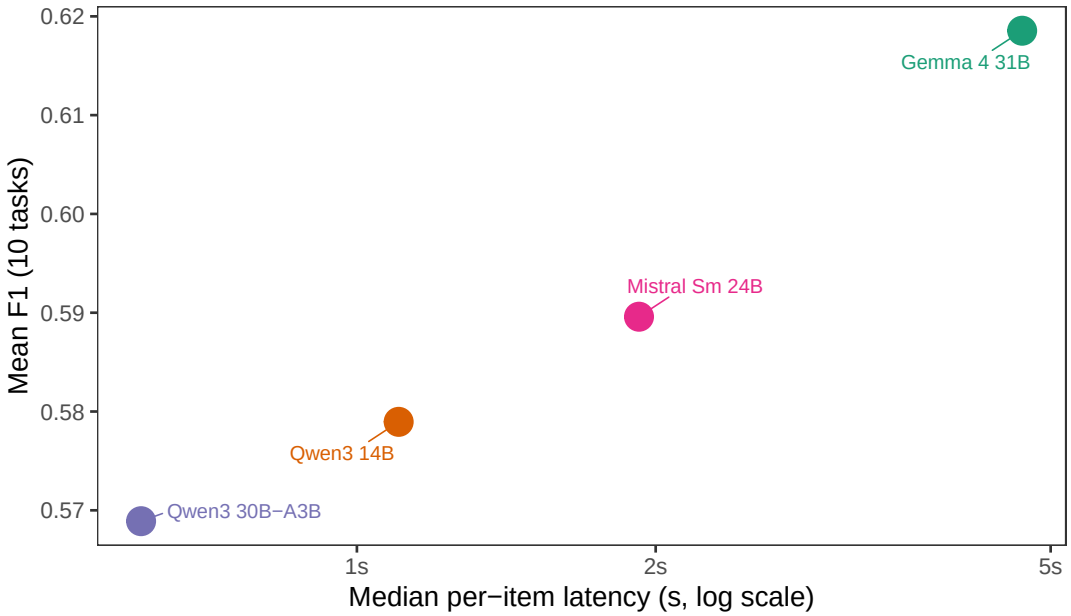

Figure 3: Mean F1 across tasks vs median per-item latency, for the four local models. API models are excluded because their per-call wall-clock mixes server load, queueing, and network round-trip and is not directly comparable to local inference time.

al. 2026). I tested batch sizes of 10 and 20 against a serial baseline (b=1). Batching helps on short prompt tasks with little or no accuracy loss (Figure 4), but creates problems on tasks with long, multi-class codebooks (12 to 50 categories), where the model already has a long instruction prompt to process on every call.

On the short prompt tasks (Gilardi relevance/stance, Ballard incivility, SemEval, SCOTUS), the six models in the batched comparison reach 1.2× to 2.7× per-item speedup at b=10. Wesleyan ad tone is the exception in the short prompt set: most local models slow down slightly under batching there. Mean-F1 changes are mostly within ±0.05, with a handful of cells (e.g., Wesleyan × Qwen3-30B-A3B at b=10) hitting larger swings. b=10 captures most of the speedup, and the move from b=10 to b=20 buys little extra throughput while raising fragility on smaller local models.

On long codebook tasks (Halterman CCC, BFRS, BES MII, CMP manifestos), two patterns emerge. Local Ollama batching does not help: the instructions are already long, so packing additional items does not save the cost the model pays to read them. A pilot cell (Halterman CCC × Gemma × b=10) ran at 14.9 s/item, slightly slower than the serial 14.1 s/item baseline. OpenAI batching looks fast in latency terms, but the rate of malformed responses climbs sharply. Figure 4 understates this problem because F1 is computed only on outputs that can be parsed. Failed responses are a separate reliability cost. gpt-5.5 batched on Halterman CCC at b=10 returns malformed output for 68% of items, and gpt-5.5 on Wesleyan at b=20 hits 48%. gpt-5.4-nano tops out at 12% on the same tasks and is more reliable under batching.

3.5 Cost

The OpenAI portion of the v2 serial run cost $22.86 for 10,000 API predictions: about $1.20 for gpt-5.4-nano and $21.66 for gpt-5.5. The Claude Sonnet 4.6 portion (5,000 predictions) ran in roughly the same cost range as gpt-5.5. Among the OpenAI tiers, gpt-5.4-nano is the cost-efficient option in this comparison, about 17 times cheaper than gpt-5.5 with a 0.021 F1 lag (0.600 vs 0.621). Local models avoid API charges.

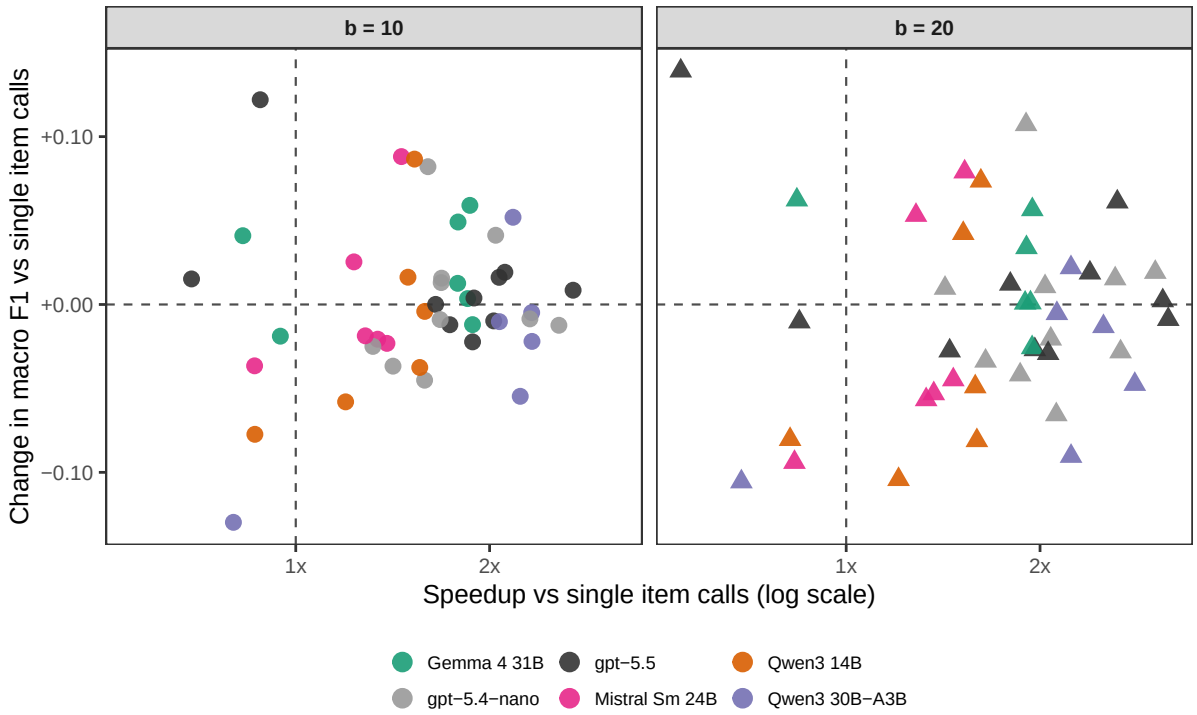

Figure 4: Per-cell speedup vs accuracy change for batched inference relative to single item (serial) calls, faceted by batch size (b=10 left, b=20 right). Each point is one (task, model) cell. Upper-right = faster and at-least-as-accurate, lower-right = fast but worse, left of x=1 = batching does not help here. Malformed output rates are not shown in this figure. They are discussed in the text.

Implications for applied use

The results imply a workflow rather than a single best model. For researchers, a small validation sample on the target task should come before production use. At this scale, the benchmark average does not reliably predict which model performs best on any specific task, and task-level reversals are common. All four local models fit on a 32 GB Apple Silicon MacBook (the largest dense model takes about 5 seconds per item, the fastest about 0.6), so a one-laptop setup is enough hardware for the full benchmark.

For metric reporting on imbalanced label sets, accuracy and MCC should appear alongside macro F1. Mellon BES MII is a clean illustration: for the top model, macro F1 is about 0.55, accuracy is 0.95, and MCC is 0.93.

For batching, b=10 is a reasonable starting point on short prompt tasks. The accuracy cost is small in this benchmark and the speedup is meaningful. For tasks with long, multi-class codebooks, my results suggest researchers should not batch aggressively unless retries are built in. Malformed output rates on gpt-5.5 reach 26 to 68 percent at b=10/20 on Halterman CCC, BFRS, and Wesleyan, and gpt-5.4-nano is the more reliable batched option for that workload.

Limitations and scope

Model selection and prompting. The model set reflects a practical applied setup, not the full set of available models: four local models that fit a 32 GB Apple Silicon setup, plus three commercial API tiers that include cheap and flagship options. I did not optimize prompts separately per model. Some are verbatim from the source paper, others are derived from published codebooks. This makes the benchmark closer to typical applied use, but it is not a maximum-performance comparison.

Task averaging and rare labels. The mean F1 ranking is descriptive. It is not usually the target quantity

for an applied coding project. Tasks differ in label count, class imbalance, construct complexity, and prompt provenance. For imbalanced codebooks, average metrics can hide failures on rare classes.

Beyond this benchmark. Local inference keeps text on the researcher's machine, which can simplify workflows with sensitive or restricted data. The benchmark studies prompt based classification only. For large labeled datasets and classification tasks with a fixed label set, fine-tuned local models, supervised classifiers, or embedding-based methods may be cheaper and more reliable. API endpoints can change behavior between releases even when the model name is stable, while local open-weight models are easier to freeze exactly.

References

Ballard, A. O., DeTamble, R., Dorsey, S., Heseltine, M., & Johnson, M. (2022). Incivility in Congressional tweets. *American Politics Research*.

Chae, Y., & Davidson, T. (2025). Large language models for text classification: From zero-shot learning to instruction-tuning. *Sociological Methods & Research*.

Gilardi, F., Alizadeh, M., & Kubli, M. (2023). ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30).

Halterman, A., & Keith, K. A. (2025). Codebook LLMs: Evaluating LLMs as measurement tools for political science concepts. *Political Analysis*.

Mellon, J., Bailey, J., Scott, R., Breckwoldt, J., Miori, M., & Schmedeman, P. (2024). Do AIs know what the most important issue is? Using language models to code open-text social survey responses at scale. *Research & Politics*.

Ornstein, J. T., Blasingame, E. N., & Truscott, J. (2025). How to train your stochastic parrot: Large language models for political texts. *Political Science Research and Methods*.

Pipal, M., Vogel, S. J., Wack, M., & Esser, F. (2026). Researchers waste 80% of LLM annotation costs by classifying one text at a time. *arXiv:2604.03684*.

Zhang, M., Cakmak, F., Neumann, M., Zimmeck, S., Oleinikov, P., Yao, J., Yu, H., Jacewicz, A., Tassone, I., Floyd, B., Baum, L., Franz, M. M., Ridout, T. N., & Fowler, E. F. (2025). Comparable 2022 general election advertising datasets from Meta and Google. *Scientific Data*.

Companion materials

Full per-task tables, malformed output tables, prompt provenance, batched inference details, summary CSVs, and reproduction instructions are in the HTML companion at `output/report.html` (and on the GitHub repository).

Tasks

Table 1: Tasks in the benchmark. 'Family' is the grouping used in the family analysis. 'Provenance': Verbatim = prompt unchanged from source paper, Verbatim-adapted = source content with format-only changes, Derived = our prompt reasoned from the published codebook. 'Prompt': short (15-35 lines) or long codebook (47-126 lines). This is the trait that determines local batching feasibility. The CMP task uses the Halterman & Keith (2025) domain-collapsed 7-class version of the CMP/MARPOR coding scheme, not the full CMP category set.

Task	Description	Source	Family	Labels	Provenance	Prompt
Gilardi relevance	Tweets, classified as about content moderation or not.	Gilardi 2023	Relevance / Incivility	binary	Verbatim	short
Ballard incivility	Tweets by U.S. members of Congress, classified for incivility.	Ballard 2022	Relevance / Incivility	binary	Derived	short
Gilardi stance	Tweets about content moderation, classified by stance toward moderation.	Gilardi 2023	Sentiment / Stance / Tone	3-class	Verbatim	short
SemEval stance	Tweets toward five named political targets, classified by stance.	Chae & Davidson 2025	Sentiment / Stance / Tone	3-class	Derived	short
SCOTUS sentiment	Tweets about U.S. Supreme Court rulings, classified by sentiment.	Ornstein et al. 2025	Sentiment / Stance / Tone	3-class	Derived	short
Wesleyan ad tone	U.S. Meta political ads, classified by tone toward a candidate.	Zhang et al. 2025	Sentiment / Stance / Tone	3-class	Derived	short
CCC protest type	News stories about U.S. protest events, classified by event type.	Halterman & Keith 2025	Event coding	4-class	Verbatim-adapted	long codebook
BFRS violence	News stories about Pakistani political violence, classified by event type.	Halterman & Keith 2025	Event coding	12-class	Verbatim-adapted	long codebook
CMP manifestos	Quasi-sentences from political party manifestos, classified by policy domain.	Halterman & Keith 2025	Policy-topic coding	7-class	Derived	long codebook
BES MII	Open-ended British Election Study responses, classified by issue topic.	Mellon et al. 2024	Policy-topic coding	50-class	Verbatim-adapted	long codebook

Per-task winners

Table 2: Top model per task by macro F1, with accuracy and MCC for the same model on the same task. CIs and the full per-(task, model) table are in the HTML companion.

Task	Top model	Macro F1	Accuracy	MCC
BES MII (50-cl)	gpt-5.5	0.547	0.946	0.934
BFRS violence (12-cl)	gpt-5.5	0.693	0.730	0.691
Ballard incivility	Claude Sonnet 4.6	0.563	0.848	0.515
CCC protest type	Qwen3 14B	0.499	0.610	0.387
CMP manifestos (7-cl)	gpt-5.5	0.597	0.646	0.545
Gilardi relevance	gpt-5.5	0.950	0.954	0.908
Gilardi stance	Claude Sonnet 4.6	0.568	0.648	0.471
SCOTUS sentiment	Qwen3 14B	0.665	0.706	0.539
SemEval 2016 stance	gpt-5.5	0.773	0.768	0.659
Wesleyan ad tone	gpt-5.4-nano	0.711	0.918	0.808

Batched malformed output rates on long codebook tasks

Table 3: Malformed output rates for the two OpenAI tiers on the long codebook tasks under batched inference. Local Ollama models and short prompt tasks are excluded. Rates there are mostly low and discussed in Section 3.4.

Task	Model	Batch size	Malformed output rate
BES MII	gpt-5.4-nano	10	0%
BES MII	gpt-5.4-nano	20	0%
BES MII	gpt-5.5	10	0%
BES MII	gpt-5.5	20	0%
BFRS Pakistan	gpt-5.4-nano	10	12%
BFRS Pakistan	gpt-5.4-nano	20	8%
BFRS Pakistan	gpt-5.5	10	28%
BFRS Pakistan	gpt-5.5	20	32%
CCC protest	gpt-5.4-nano	10	6%
CCC protest	gpt-5.4-nano	20	8%
CCC protest	gpt-5.5	10	68%
CCC protest	gpt-5.5	20	40%
CMP manifestos	gpt-5.4-nano	10	0%
CMP manifestos	gpt-5.4-nano	20	0%
CMP manifestos	gpt-5.5	10	10%
CMP manifestos	gpt-5.5	20	0%
Wesleyan ads	gpt-5.4-nano	10	0%
Wesleyan ads	gpt-5.4-nano	20	0%
Wesleyan ads	gpt-5.5	10	26%
Wesleyan ads	gpt-5.5	20	48%

Performance under alternative metrics

Table 4: Cross-task means of macro F1, accuracy, and MCC, by model. The top three models are unchanged across all three metrics; MCC and macro F1 give a Spearman rank correlation of 0.93 across the seven models, while accuracy reorders the lower half slightly because some imbalanced tasks have high chance accuracy.

Model	Mean macro F1	Mean accuracy	Mean MCC
gpt-5.5	0.621	0.750	0.608
Claude Sonnet 4.6	0.620	0.734	0.594
Gemma 4 31B	0.619	0.746	0.600
gpt-5.4-nano	0.600	0.731	0.584
Mistral Sm 24B	0.590	0.726	0.561
Qwen3 14B	0.579	0.744	0.568
Qwen3 30B-A3B	0.569	0.735	0.548